

Contrast, equalization and quantization

1. LINEAR CONTRAST CHANGES

In this exercise, we worked with the provided low-contrast grayscale image. First, we increased its global contrast in order to make the range of intensities wider and obtain a better starting point for the following transformations. After that, we applied five different linear contrast operations, each one modifying the gray levels in a different way.

1.0.1. (a) Expansion of the dark tones $[0, 0.4]$

In this first transformation, we expanded the dark intensity values contained in the interval $[0, 0.4]$ so that they covered the whole output range $[0, 1]$. In practice, this makes the differences between dark tones much more visible. Pixels with very low intensities remain dark, while pixels closer to 0.4 become much brighter. As a result, the details located in the dark regions of the image are emphasized, although brighter regions tend to become saturated.



Figure 1: Expansion of the dark tones in the interval $[0, 0.4]$.

1.0.2. (b) Expansion of the light tones $[0.6, 1.0]$

For this case, we expanded the light intensity values in the interval $[0.6, 1.0]$ to the full output range $[0, 1]$. This operation focuses only on the brightest areas of the image. Since the original image is mostly dark, only a small portion of the pixels belongs to this interval. Therefore, the resulting image becomes almost completely black, except for some small bright details that are preserved and stretched.



Figure 2: Expansion of the light tones in the interval $[0.6, 1.0]$.

1.0.3. (c) Contrast inversion: negative image

In this transformation, we computed the negative of the contrast-enhanced image. Since the image intensities were normalized in the interval $[0, 1]$, the operation was carried out by replacing each pixel value x with $1 - x$. This means

Assignment Lab 3

that dark areas become bright, bright areas become dark, and intermediate gray values remain in intermediate positions. The resulting image clearly shows the contrast inversion effect.



Figure 3: Negative image obtained by inverting the gray levels.

1.0.4. (d) Clipping of intensity levels above 0.7

In this part, we applied clipping to all intensity values greater than 0.7. Any pixel whose value was above that threshold was limited to exactly 0.7, while the rest of the image remained unchanged. Since the input image does not contain a large number of very bright pixels, the visual difference is not extremely strong. However, the operation reduces the intensity of the brightest regions and prevents them from reaching the maximum white level.



Figure 4: Clipping of intensity values above 0.7.

1.0.5. (e) Binarization at level 0.5

Finally, we binarized the image using a threshold value of 0.5. Pixels with intensity lower than 0.5 were assigned the value 0, corresponding to black, while pixels with intensity greater than or equal to 0.5 were assigned the value 1, corresponding to white. This transformation removes all intermediate gray levels and produces a purely black-and-white image. The result highlights which areas of the image are darker or brighter with respect to the chosen threshold.



Figure 5: Binarization of the image using a threshold of 0.5.

2. NONLINEAR CONTRAST CHANGES

2.1. Discussion of gamma's value

Gamma correction is a non-linear function that takes place in the mathematical architecture of an image and its function is to map intensities. It is computed after the original optical signal has been convolved with the lens's Point Spread Function (PSF), representing the actual light hitting the sensor, $g(K * O)$ in Equation 1.

$$u = Q\{\Pi g(K * O) + n\} \quad (1)$$

Answering to the question, if gamma is less than 1, the function will be concave, which means that dark tonalities are expanded and light tonalities are compressed, resulting in increased contrast in the shadows. On the other hand, if gamma is greater than 1, the function will be convex, which means that dark tonalities are compressed and light tonalities are expanded. In this case, the result will be a decrease in shadow contrast, but an increase in highlight contrast. In Figure 6, we can see the graphical representation of the gamma correction function for both cases, where the x-axis represents the input light levels and the y-axis represents the output light levels after applying gamma correction.

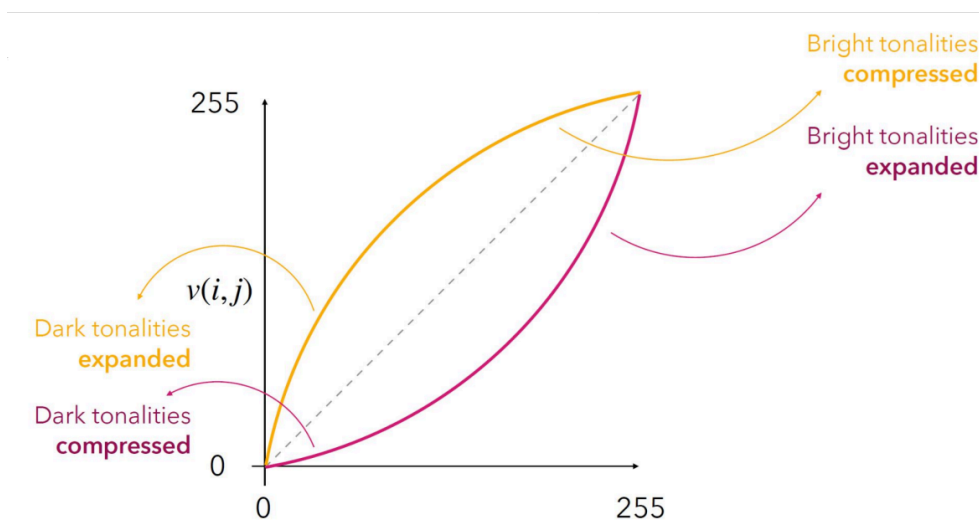


Figure 6: Graphical representation of the gamma correction function for gamma < 1 (purple line) and gamma > 1 (yellow line).

Assignment Lab 3

There are two examples that clearly explain this behaviour. The first one is to look at the graphical representation (Figure 7) of the function with gamma greater than 1, where we can see how a light level input is expanded. This graphical representation of the gamma correction is given in the theoretical slides.

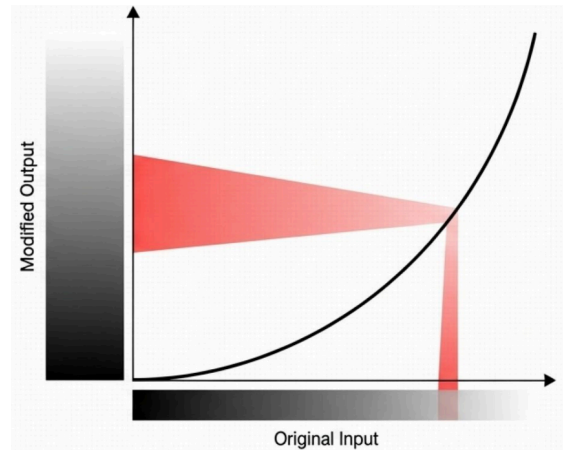


Figure 7: Graphical representation of the gamma correction function with gamma > 1 for a light level input.

The other example is to mathematically demonstrate how gamma correction alters image contrast, we use normalized pixel values (scaled from 0 to 1) and the following power-law equation:

$$V_{\text{out}} = V_{\text{in}}^{\gamma} \quad (2)$$

If we consider a dark input pixel with an intensity of 20% ($V_{\text{in}} = 0.2$), which being less than 0.5 would be more darker than brighter and a gamma encoding value of $\gamma = 0.5 < 1$:

$$V_{\text{out}} = 0.2^{0.5} \approx 0.447 \quad (3)$$

The output intensity is elevated to approximately 45%. Because a narrow range of dark input values (0 to 0.2) is mapped to a significantly wider range of output values (0 to 0.447), the dark tonalities are effectively expanded.

2.2. Nonlinear contrast change on a low contrast image

For this section we will modify the contrast of the image “img/low_contrast.png” (Figure 8) using the gamma correction function `imadjust` which allows you by adding a final argument, gamma, to perform nonlinear contrast changes.



Figure 8: Original low contrast image.

Assignment Lab 3

After testing different values of gamma (Figure 9) plotting the image and its histogram, we found that a value of would be in the range of 1.4 to 1.5, which is a value greater than 1, so the function will be convex and the dark tonalities will be compressed and light tonalities will be expanded. This way we can visualize the dark details of the image without decreasing too much the overall contrast of the image.

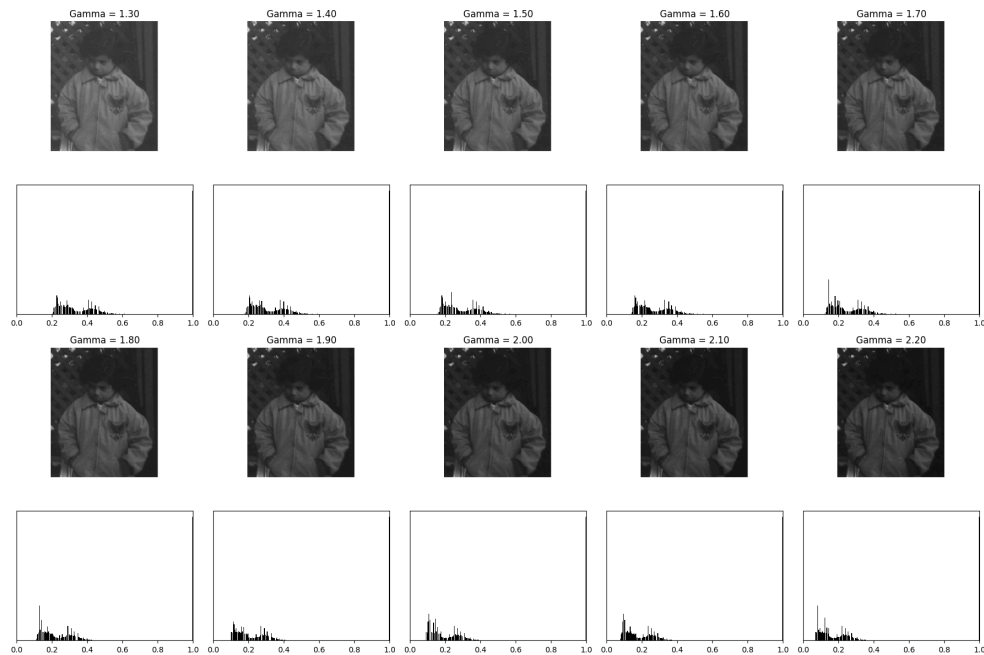


Figure 9: Low contrast image after applying a nonlinear contrast change with $\gamma = [1.3, 2.2]$.

To be more precise, we analyzed ten values of gamma from 1.4 to 1.5 with a step of 0.01 and we found that the best value was 1.41.

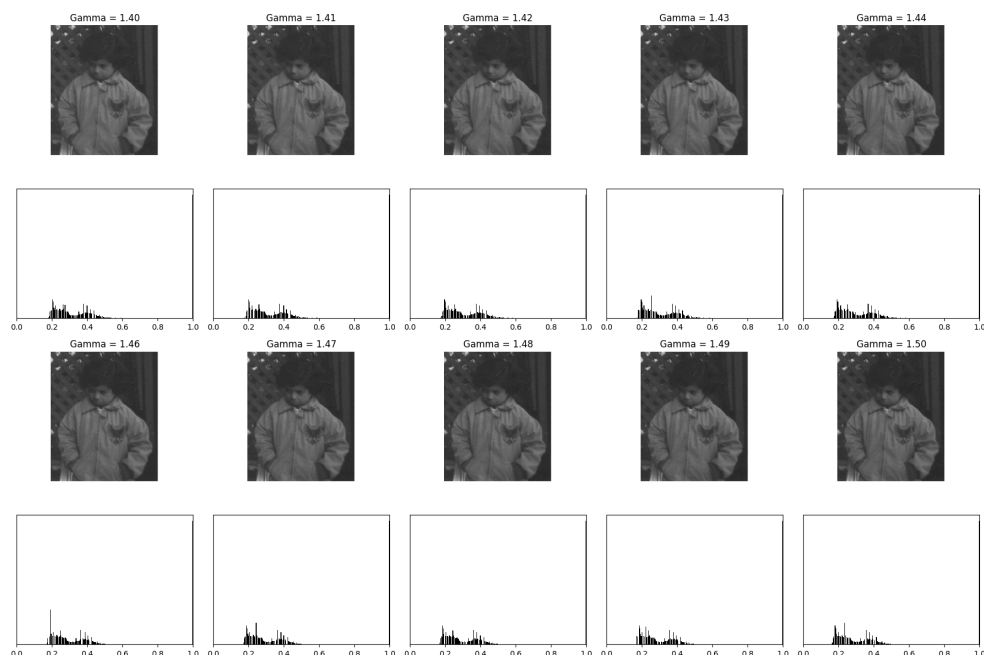


Figure 10: Low contrast image after applying a nonlinear contrast change with $\gamma = 1.41$.

Assignment Lab 3

For this particular parameter value, the main of the dark pixels (which would include information about shadows such as hair and fabrics) is optimally placed between pixel intensities of 0.15 and 0.25. This ensures excellent contrast between the dark areas and the brighter areas while keeping the data sufficiently above the 0.0 minimum level. Increasing the value of gamma to 1.48 or 1.50 would place this particular peak too close to the 0.0 value and result in compression and crushing of blacks, which would mean destruction of any texture present.

The final comparison between the low contrast image and the final result would be the following figure (Figure 11).

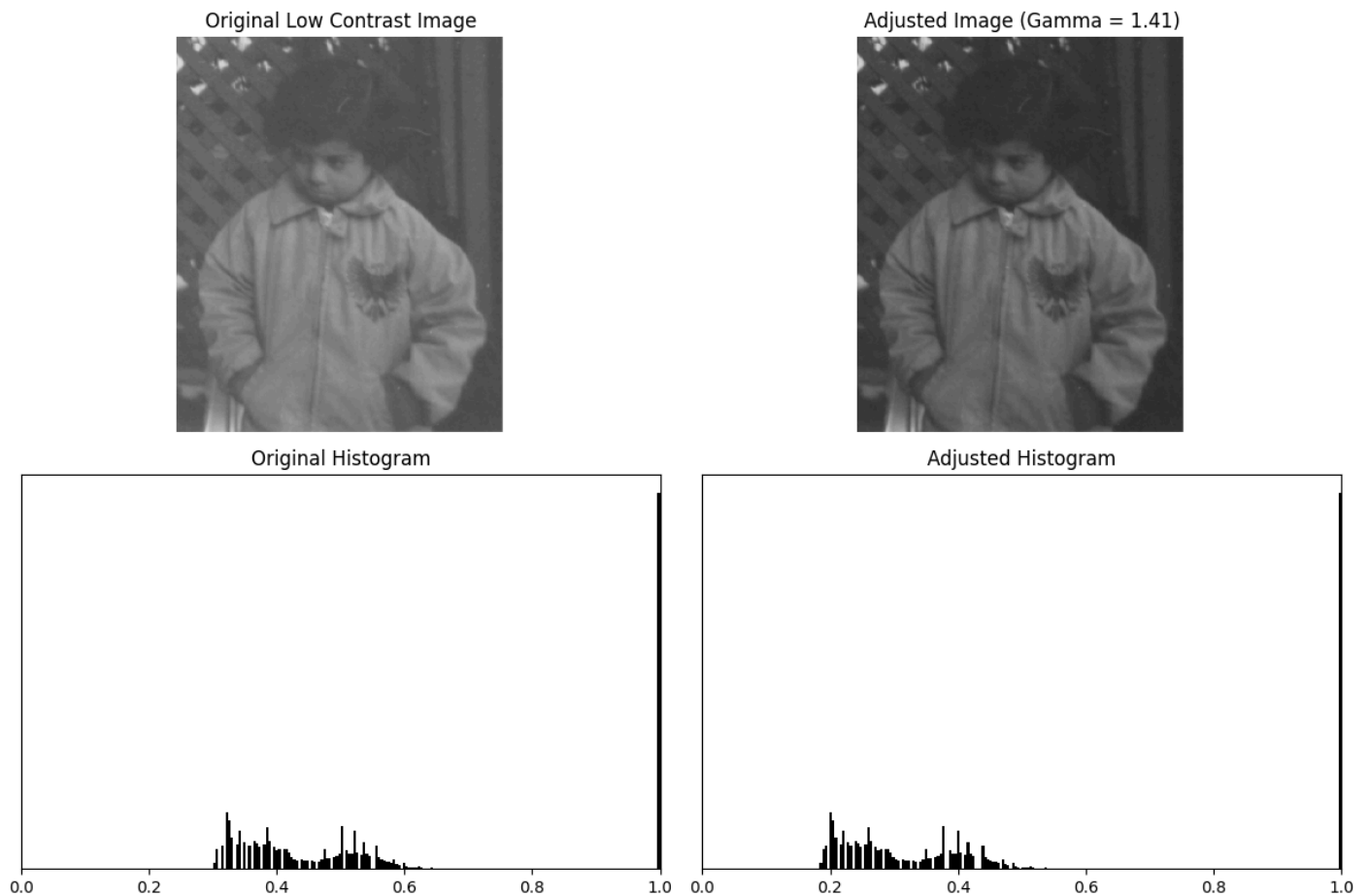


Figure 11: Final comparison between the low contrast image and the gamma-corrected result.

3. Image Histogram Equalization

3.1. Display of Histograms

To display the histograms of a RGB and grayscale image we have chosen the images of “img/lena_color.png” and “img/lena.png” respectively (Figure 12 and Figure 13).

Assignment Lab 3

Figure 12: Original RGB image.



Figure 13: Original grayscale image.

Thanks to the function `plt.hist(image.ravel(), bins = 256)` found in the given notebook's code, we can display the histograms of both images (Figure 14 and Figure 15).

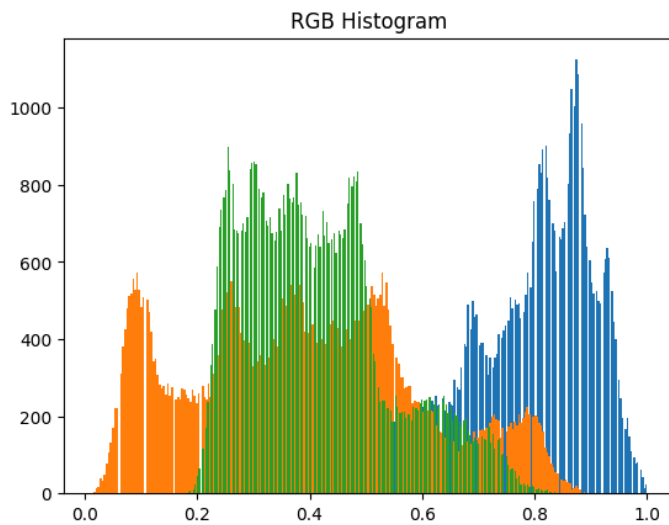


Figure 14: Histogram of the RGB image separated into R, G and B channels.

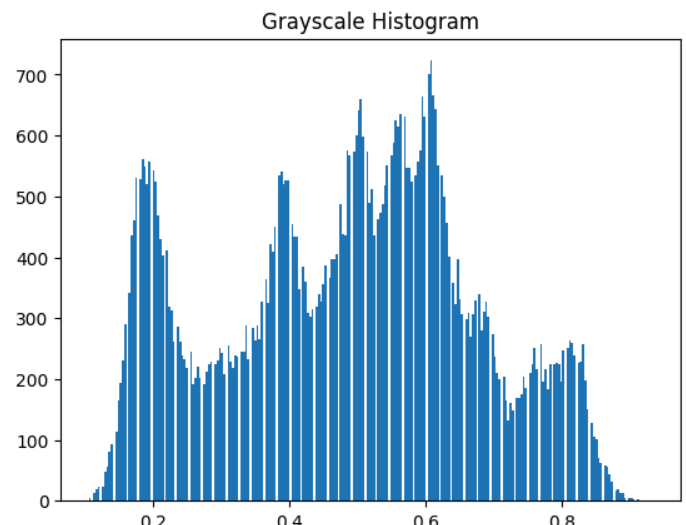


Figure 15: Histogram of the grayscale image.

As is precised in the caption of Figure 14, the histogram of the RGB image is separated into R, G and B channels, where we can see that the red channel has a higher frequency of pixel intensities in the range of 100 to 200, while the green and blue channels have a more uniform distribution of pixel intensities. On the other hand, the histogram of the grayscale image shows a more uniform distribution of pixel intensities across the entire range from 0 to 255, with a slight peak around the intensity value of 128. This indicates that the grayscale image has a more balanced distribution of light and dark pixels compared to the RGB image.

3.2. Image Equalization

By introducing the histogram equalization is a technique used to enhance the contrast of an image by redistributing the pixel intensity values. The goal is to achieve a more uniform distribution of pixel intensities across the entire range, which can help to reveal details that may be hidden in the original image.

Assignment Lab 3

To perform histogram equalization, we must remind that the formula to compute the new pixel intensity values is given by:

$$h(v) = \text{round}\left(\frac{\text{cdf}(v) - \text{cdf}_{\min}}{N - \text{cdf}_{\min}} \times (L - 1)\right) \quad (4)$$

Where $h(v)$ is the new pixel intensity value, $\text{cdf}(v)$ is the cumulative distribution function of the pixel intensity values, $\text{cdf}_{\min}$ is the minimum value of the cumulative distribution function, N is the total number of pixels in the image, and L is the number of possible intensity levels (usually 256 for an 8-bit image).

Translating Equation 4 into python code, we can implement the histogram equalization function as follows:

```
import numpy as np
import matplotlib.pyplot as plt

def equalize_histogram(image):

    # 1. Calculate the histogram (256 bins for 8-bit image)
    hist, bins = np.histogram(image.flatten(), bins=256)

    # 2. Calculate the Cumulative Distribution Function (CDF)
    cdf = hist.cumsum()

    # The Equalization Formula
    L = 2**8 # Number of possible intensity levels (256)
    cdf = (cdf - cdf.min()) * (L - 1) / (cdf.max() - cdf.min())
```

After implementing the histogram equalization function, we can apply it to several images and compare the results. For example, we can apply it to the girl low contrast image (Figure 8) and to more low contrast images found in the images folder like Figure 16 and Figure 17 and then display the results along with their histograms.

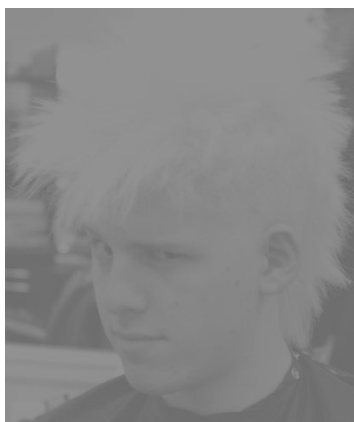


Figure 16: Original low contrast image of Gabe.



Figure 17: Original low contrast image of a hat.

We will compare the histogram equalization results image by image. Starting with the girl we can see the comparison image in Figure 18, where we can see that the histogram equalization has enhanced the contrast of the image, making the details in the shadows and highlights more visible. The histogram of the equalized image shows a more uniform

Assignment Lab 3

distribution of pixel intensities across the entire range compared to the original image, which had a more concentrated distribution of pixel intensities in the mid-range.

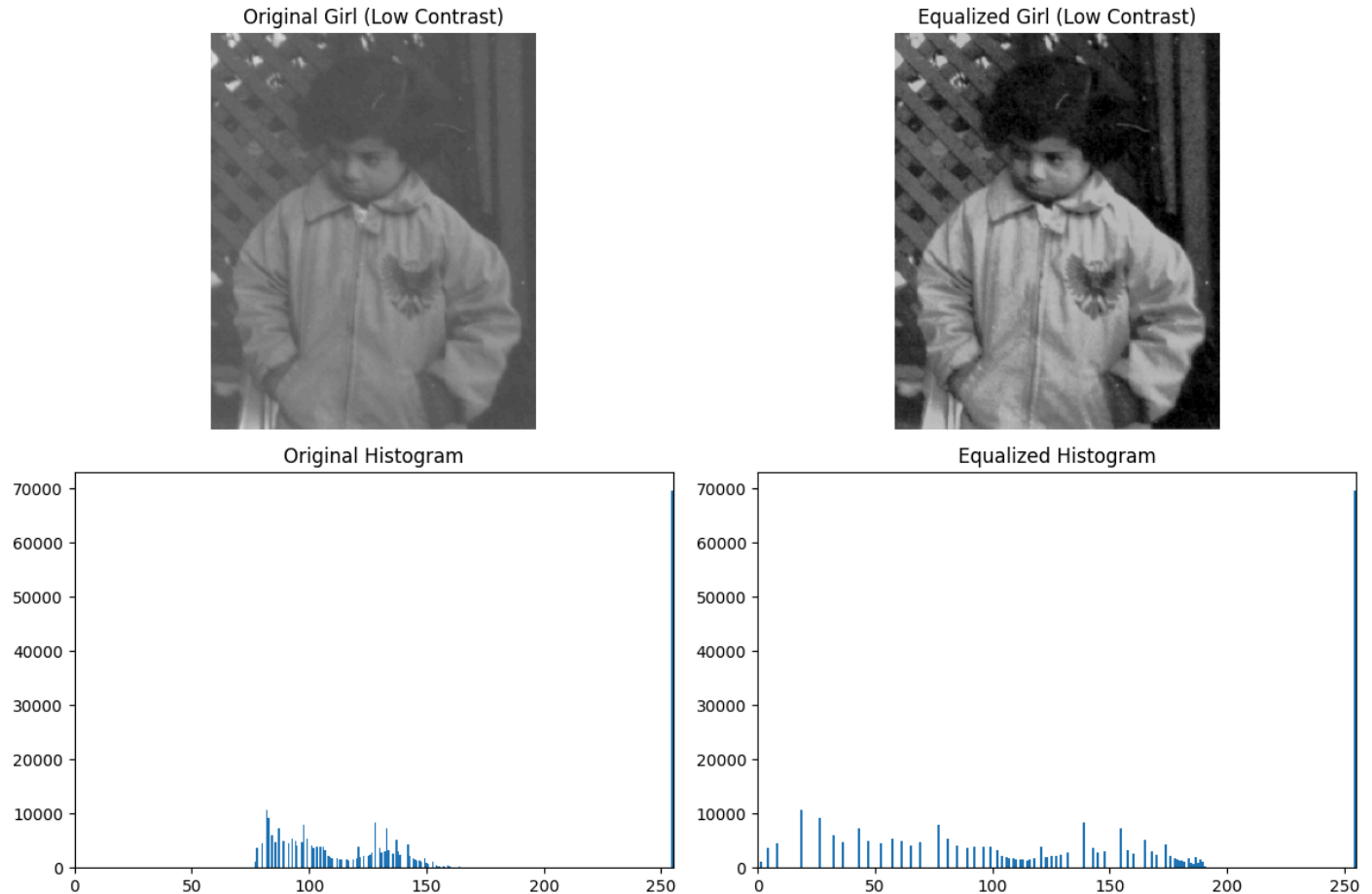


Figure 18: Comparison between the original low contrast image of the girl and the histogram equalized image.

When it comes to the Gabe image, it is also possible to make a comparison using Figure 19. It is clear that, despite the equalization process taking place, the visual result achieved is not even close to being similar to the one obtained for the girl. Although the mathematical process caused the pixel intensities to be spread out all the way from zero to maximum, the visual outcome was not natural at all; on the contrary, it was very noisy.

The reason behind the failure of the equalization process in the Gabe case can be easily found in the histograms. The problem is that the original image contained very few pixels, which were clustered together in a tiny distribution. When trying to stretch this distribution across the whole spectrum of pixel intensities, it resulted in huge gaps between individual pixels in the histogram.

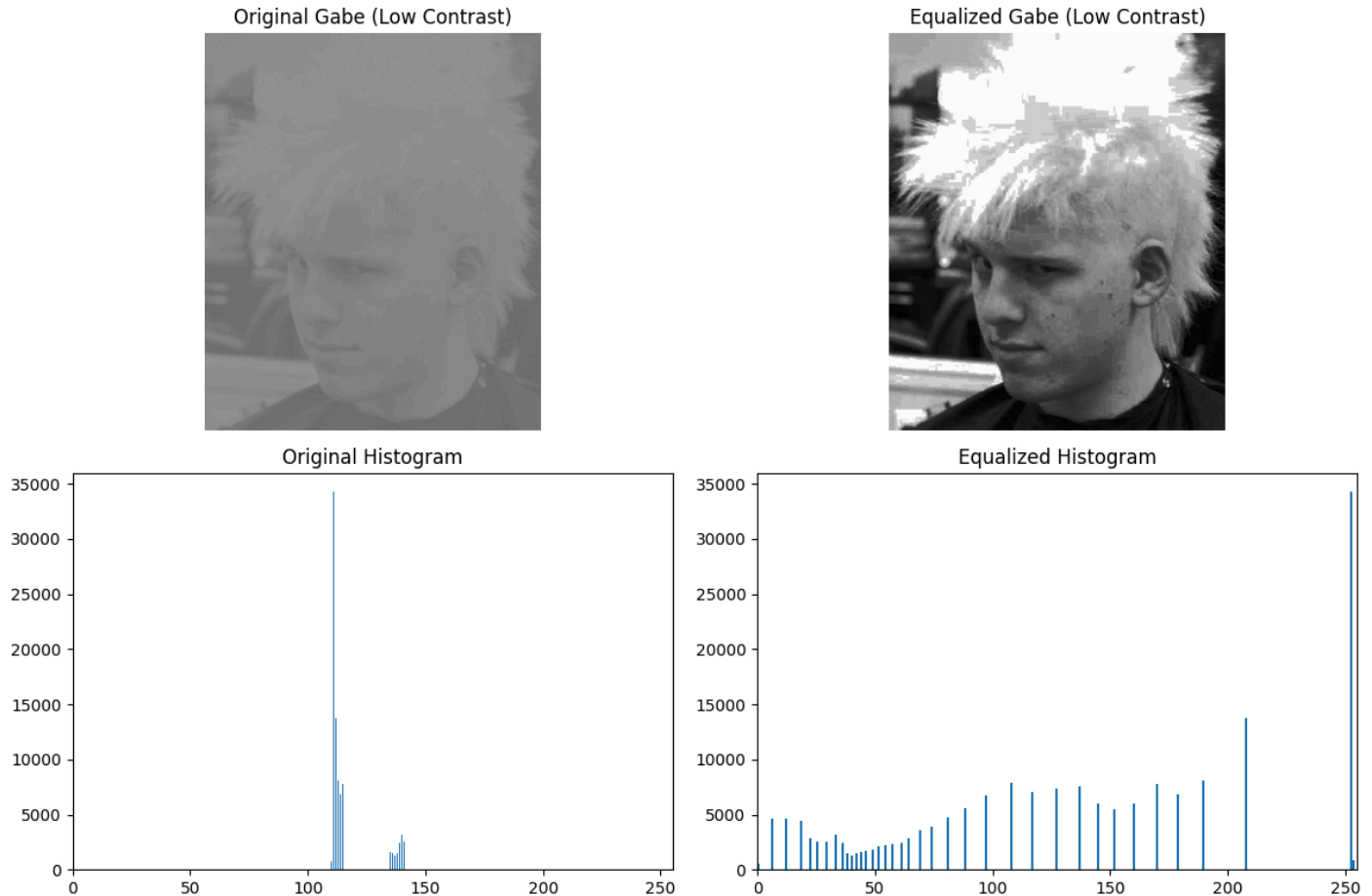
Assignment Lab 3

Figure 19: Comparison between the original low contrast image of Gabe and the histogram equalized image.

Finally, we analyze the woman dressing the hat, depicted in Figure 20, which can be considered the most successful example of histogram equalization amongst all the images analyzed. It is a very realistic result, with detail like the freckles in the textures being clearly visible and well-preserved.

This is owed entirely to the intrinsic structure of pixel data in this particular image. Firstly, the image is formed almost entirely of dense, high-frequency textures such as the shapes of the face and the clothing, and does not contain any large areas of uniformly colored background. Consequently, its histogram does not feature the large spikes in pixel density that normally disrupt and distort the Cumulative Distribution Function (CDF).

As a result, the equalization algorithm is able to evenly stretch the compressed pixel densities across the whole dynamic range without introducing any quantization or clipping artifacts. Given that the underlying image data is sufficiently continuous and dense, there are no sudden discontinuities introduced into the histogram through the process of stretching.

Assignment Lab 3

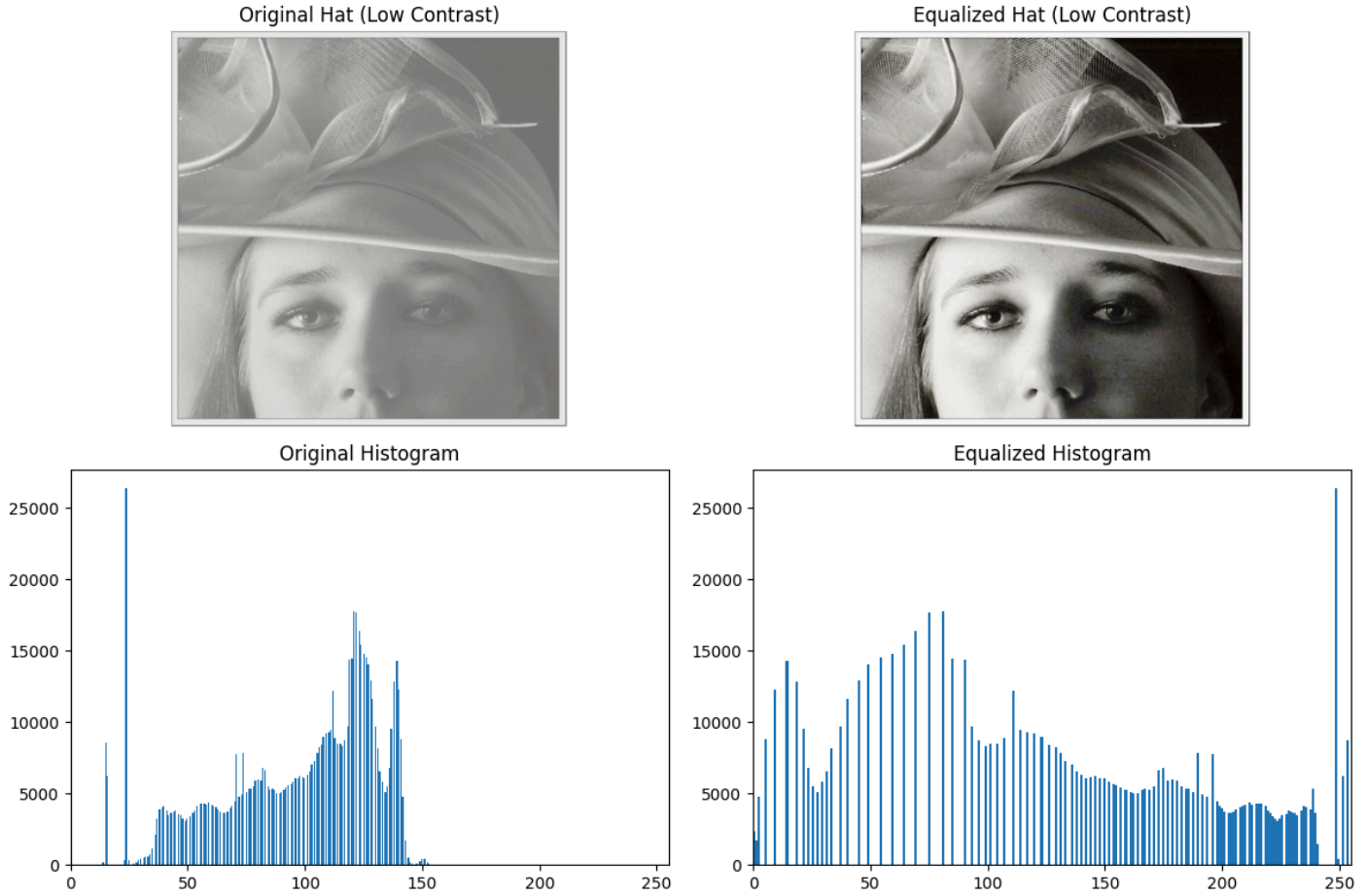


Figure 20: Comparison between the original low contrast image of the hat and the histogram equalized image.

4. Image Entropy

In this exercise, we implemented a function to calculate the entropy of an image and used it to compare an original grayscale image with its equalized version. Entropy is a quantitative measure of the amount of information or variability contained in the gray-level distribution of an image. In other words, it indicates how spread out the pixel intensities are among the different possible levels.

The entropy of an image is defined as:

$$H = - \sum_{i=0}^{255} p_i \log_2(p_i) \quad (5)$$

where p_i is the probability of occurrence of the gray level i . To compute this value, we first calculated the histogram of the image using 256 bins, corresponding to all possible intensity values between 0 and 255. Then, we normalized the histogram by dividing each frequency by the total number of pixels, obtaining the probability distribution of the gray levels.

After that, we applied the entropy formula. In the implementation, we excluded the probability values equal to zero before calculating the logarithm, since $\log_2(0)$ is not defined. This does not alter the entropy result because gray levels that do not appear in the image do not contribute to the sum.

Assignment Lab 3

Once the function was implemented, we calculated the entropy of the original image and of the same image after histogram equalization. The equalization process redistributes the gray levels so that the contrast becomes more balanced across the available intensity range. As a consequence, the equalized image usually presents a wider and more uniform use of the possible gray values, which can lead to a higher entropy.

Visually, the equalized image shows stronger contrast and reveals details that were less noticeable in the original image. This happens because the gray levels that were previously concentrated in a narrower range are spread over a larger interval. Therefore, the resulting image contains a richer intensity distribution, which is consistent with the entropy comparison obtained from our implementation.



Figure 21: Comparison between the original image and its equalized version, including the corresponding entropy values.

5. Quantization

5.1. Experimental smallest quantization levels

As is detailed in the statement of the exercise, experimentally we have evaluated Gray Lena image Figure 13 with different levels of quantization, the function given in the notebook `uniform_quantizer(image, N)`, where `N` is the number of quantization levels, has helped to perform the quantization process.

We consider the following quantization levels: 128, 64, 32, 16, 8 and 4 in Figure 22. After applying the quantization, we can visually analyze that from this quantization levels, the smallest that is unnoticeable is 32 levels. 16 levels already shows a significant loss of detail, mostly on the background, while 32 levels still retains most of the visual information and details without introducing significant artifacts. Therefore, we can conclude that the smallest number of quantization levels for which the quantization is unnoticeable for the eye is approximately 32 levels.

Assignment Lab 3

Figure 22: Quantization of the grayscale Lena image with different levels of quantization.

5.2. Binarization of an image

To binarize an image, we can use the `uniform_quantizer` function with $N=2$, which will quantize the image into two levels: 0 and 255. This effectively creates a binary image where pixels are either black (0) or white (255) based on a threshold determined by the quantization process. For example, if we apply this to the grayscale Lena image (Figure 13), we can obtain the binarized version of the image (Figure 23).



Figure 23: Binarized version of the grayscale Lena image using uniform quantization with $N=2$.

Assignment Lab 3

6. Quantization

In this exercise, we created a pop-art composition inspired by Andy Warhol. The goal was to start from an image quantized into only three intensity levels, replace those three gray levels with different RGB color palettes, and finally combine four colored versions of the same image into a single composition with double the original width and height.

6.0.1. Original image

We began with the selected input image (Figure 8) and converted it to grayscale when necessary. Working with a grayscale image is important in this exercise because the recoloring process is based on intensity levels rather than on the original colors. Each pixel is therefore represented by a single brightness value between 0 and 1.

6.0.2. Quantization into 3 gray levels

The first required step was to quantize the grayscale image into exactly three levels. Instead of keeping a continuous range of intensities, we grouped all pixel values into three categories:

- dark regions,
- medium-intensity regions,
- bright regions.

In our implementation, we assigned each pixel to one of three discrete indices: 0, 1, or 2. These indices correspond to the three gray-level classes that will later be replaced by colors. Conceptually, the quantization can be understood as dividing the normalized intensity interval $[0, 1]$ into three parts.

After quantization, the image loses smooth grayscale transitions and becomes visually simpler, with only three possible intensity values. This simplification is essential for producing the strong posterized appearance characteristic of pop-art compositions.



Figure 24: Image quantized into three intensity levels.

6.0.3. Color palette mapping

Once the image had been quantized, we used the three quantization levels as labels to apply color palettes. Each gray-level index was replaced by one RGB color from a given palette:

- level 0 was mapped to the first RGB color,

Assignment Lab 3

- level 1 was mapped to the second RGB color,
- level 2 was mapped to the third RGB color.

The laboratory statement provided four different palettes, each containing three RGB colors. We applied one palette to each copy of the quantized image. This generated four versions of the same image with identical shapes and intensity regions, but with completely different chromatic appearances.

This mapping preserves the structure of the original image while producing a much more expressive and stylized visual result. Dark, medium, and bright areas remain separated, but instead of appearing as gray tones, they become strongly contrasted colored regions.

6.0.4. Construction of the final Warhol composition

After producing the four recolored versions, we arranged them into a 2×2 grid:

- the first palette was used in the upper-left image,
- the second palette was used in the upper-right image,
- the third palette was used in the lower-left image,
- the fourth palette was used in the lower-right image.

Since the same original image is placed twice horizontally and twice vertically, the final composition has double the original width and double the original height. This fulfills the requirement of generating an output image with four duplicated colored versions of the quantized image.

The final result resembles an Andy Warhol-inspired pop-art poster: the same visual content is repeated several times, but each version uses a different high-contrast palette, producing a colorful and decorative composition.

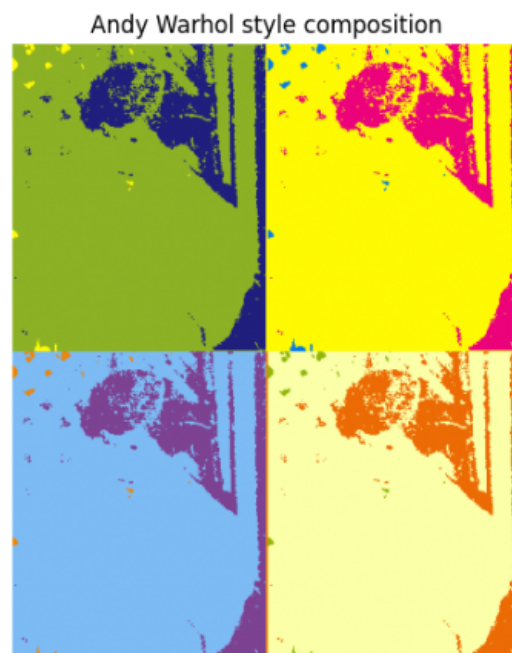


Figure 25: Final Andy Warhol-style composition obtained by recoloring the 3-level quantized image with the four RGB palettes provided in the exercise.

Assignment Lab 3

7. Quantitative Criteria of Fidelity

In this exercise, we evaluated quantitatively how much information is lost when an image is modified through uniform quantization. Instead of relying only on visual inspection, we computed two fidelity metrics between the original image and several quantized versions of it:

- the least-squares error, denoted as σ_{LS} ,
- the Peak Signal-to-Noise Ratio, denoted as PSNR.

The exercise was carried out using the original grayscale image and three quantized versions obtained with different numbers of gray levels. In our case, we compared the original image with quantizations of 3, 11, and 58 levels.

7.0.1. Least-squares error: σ_{LS}

The first metric we implemented was the least-squares error. It measures the average squared difference between the intensity of each pixel in the original image and the corresponding pixel in the quantized image.

We computed it as:

$$\sigma_{LS} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (u(i, j) - v(i, j))^2 \quad (6)$$

where:

- $u(i, j)$ is the gray level of the original image at pixel (i, j) ,
- $v(i, j)$ is the gray level of the quantized image at the same pixel,
- $M \times N$ is the total number of pixels in the image.

This quantity becomes larger when the quantized image differs more from the original one. Therefore, a high value of σ_{LS} indicates a stronger degradation, whereas a low value indicates a better approximation.

7.0.2. Peak Signal-to-Noise Ratio: PSNR

The second metric we implemented was the PSNR, which is commonly used to evaluate the fidelity of reconstructed or compressed images. It is expressed in decibels and was computed from the least-squares error as:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\sigma_{LS}} \right) \quad (7)$$

The value 255 appears because the images are represented with gray levels in the range $[0, 255]$. The PSNR increases when the error decreases. Therefore:

- a high PSNR means that the quantized image is close to the original,
- a low PSNR means that the distortion introduced by quantization is more significant.

7.0.3. Visual and numerical comparison

The figure below shows the original image together with the three quantized versions used in our analysis. The first quantization, with only 3 gray levels, produces a very strong posterization effect and a clear loss of details. With 11 levels, the image becomes noticeably more faithful to the original, although some discontinuities in the gray transitions are still visible. Finally, the version with 58 levels is visually much closer to the original image, since the quantization steps are considerably smaller.

Assignment Lab 3

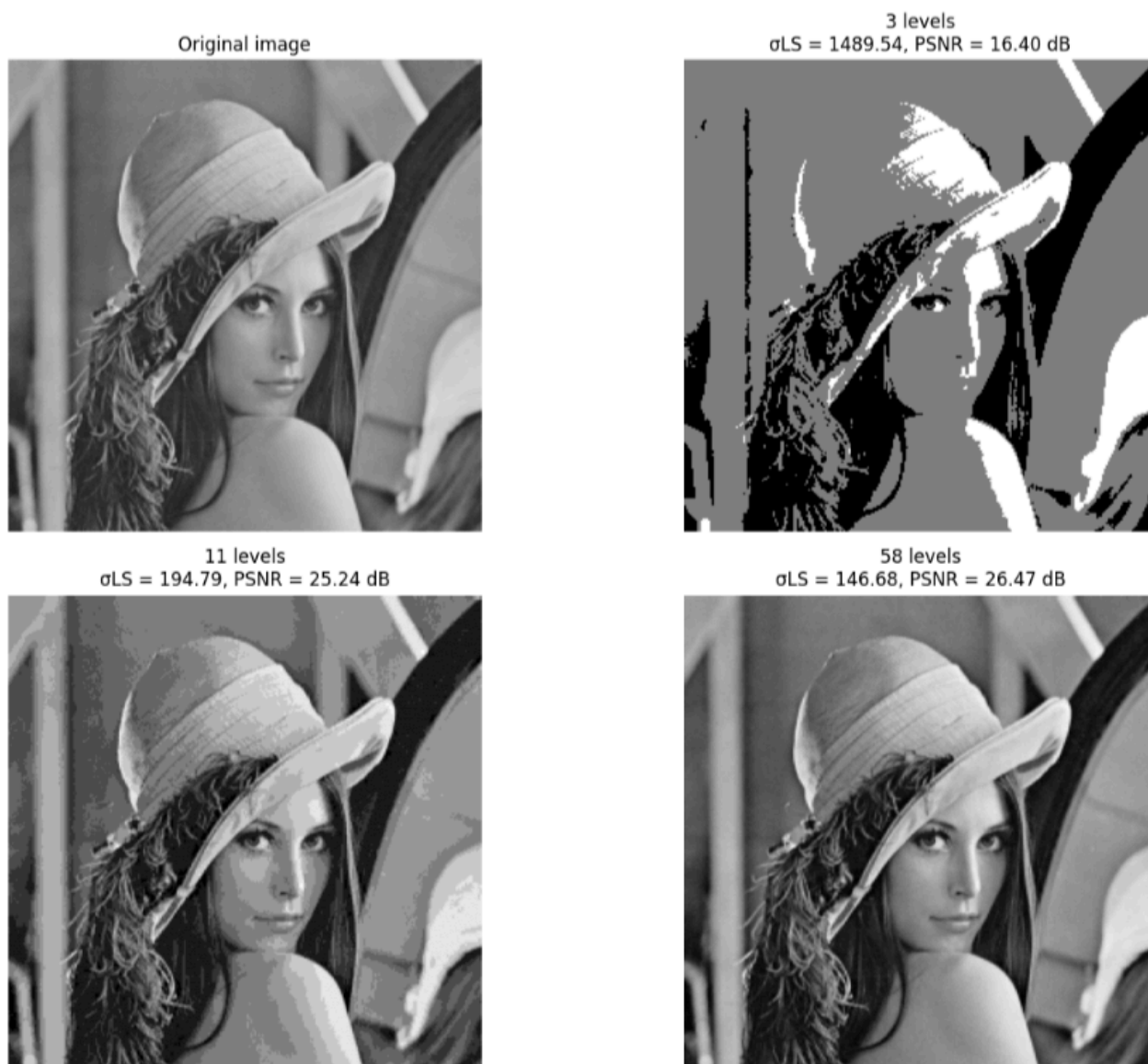


Figure 26: Comparison between the original image and its uniformly quantized versions with 3, 11, and 58 gray levels.

The numerical values obtained for the fidelity criteria can be summarized as follows:

Number of levels	σ_{LS}	PSNR (dB)
3	1489	16.40
11	194.7	25.23
58	146.68	26.48

The results confirm the expected behavior. When only 3 quantization levels are used, the approximation is very rough, so the squared error is the largest and the PSNR is the smallest. This matches the strong visual degradation observed in the corresponding image.

Assignment Lab 3

When the number of levels is increased to 11, the quantized image preserves more information from the original. As a consequence, σ_{LS} decreases and the PSNR increases. The improvement is also visible in the image, where facial details and smooth intensity transitions are better preserved.

With 58 levels, the quantized image becomes much more similar to the original one. The error decreases further and the PSNR reaches its highest value among the tested cases. Visually, the differences are much less noticeable, which agrees with the numerical measurements.

Therefore, both metrics show the same conclusion: increasing the number of quantization levels improves the fidelity of the reconstructed image. The least-squares error decreases, while the PSNR increases, indicating that the quantized image approaches the original more accurately.

8. Halftoning

8.1. Halftoning Analysis

To complete the halftoning analysis, we applied the Floyd-Steinberg error diffusion algorithm to our test images as an alternative to standard binarization. Unlike simple two-level quantization, which forces pixels into absolute black or white and creates harsh, visible banding (false contouring), the Floyd-Steinberg method distributes the mathematical rounding error to adjacent pixels. This creates a stippled dot pattern that successfully tricks the human visual system into perceiving smooth continuous gradients. In the Lena image (Figure 27), this algorithm beautifully preserves the soft tonal transitions of the skin and background using solely binary pixels. Conversely, the halftoned result for the hat image (Figure 28) behaves exactly as mathematically expected given its prior equalization. Because the previous equalization step severely clipped the highlights on the subject's face, those pure white regions generate no quantization error to diffuse, leaving them completely devoid of dots. The algorithm concentrates its stippling entirely in the textured midtones of the background and clothing, proving that it faithfully translates the extreme contrast of the underlying source matrix without introducing standard quantization artifacts.



Figure 27: Halftoned version of the grayscale Lena image using the Floyd-Steinberg error diffusion algorithm.

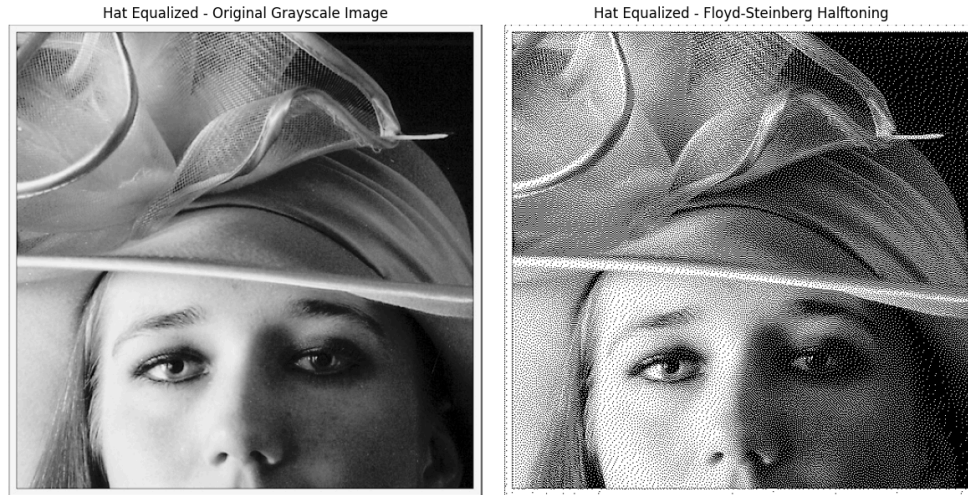
Assignment Lab 3

Figure 28: Halftoned version of the equalized hat image using the Floyd-Steinberg error diffusion algorithm.

8.2. Floyd-Steinberg Algorithm Explanation

The Floyd-Steinberg Algorithm is a type of halftoning algorithm used for images that involves the use of error diffusion to create the impression of continuous gradients by using binary pixels. Unlike thresholding where the pixel values are either rounded off to full black or full white while discarding the remaining fraction, the Floyd-Steinberg Algorithm computes the actual quantization error of the pixel value. In the process, the algorithm disperses the error fraction to unprocessed neighboring pixels. More specifically, the error is dispersed by $\frac{7}{16}$ to the right, $\frac{3}{16}$ to the bottom left, $\frac{5}{16}$ to the bottom, and $\frac{1}{16}$ to the bottom right. In this way, the total brightness of the area within the image is preserved, thereby avoiding any banding effects due to uniform quantization. The fundamental concept of error diffusion is illustrated below:

Pseudocode:

```
for each y from top to bottom:
  for each x from left to right:
    old_pixel = image[x, y]
    new_pixel = find_closest_palette_color(old_pixel)
    image[x, y] = new_pixel
    error = old_pixel - new_pixel

    // Diffuse the error to adjacent pixels
    image[x + 1, y] = image[x + 1, y] + error * (7 / 16)
    image[x - 1, y + 1] = image[x - 1, y + 1] + error * (3 / 16)
    image[x, y + 1] = image[x, y + 1] + error * (5 / 16)
    image[x + 1, y + 1] = image[x + 1, y + 1] + error * (1 / 16)
```